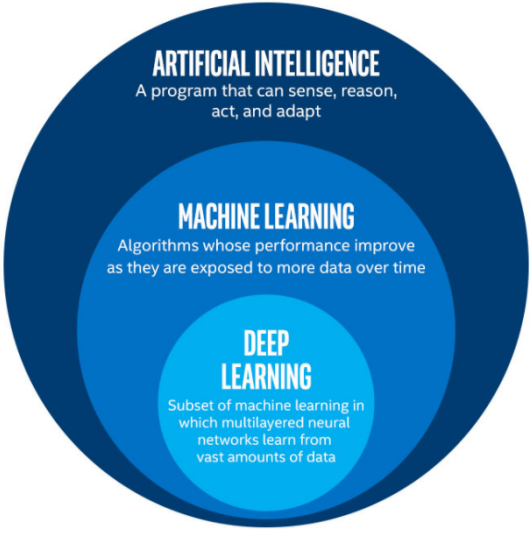
[인공지능]

Perceptron & Artificial Neural Networks

Eunsong Kang
eskang@kangwon.ac.kr

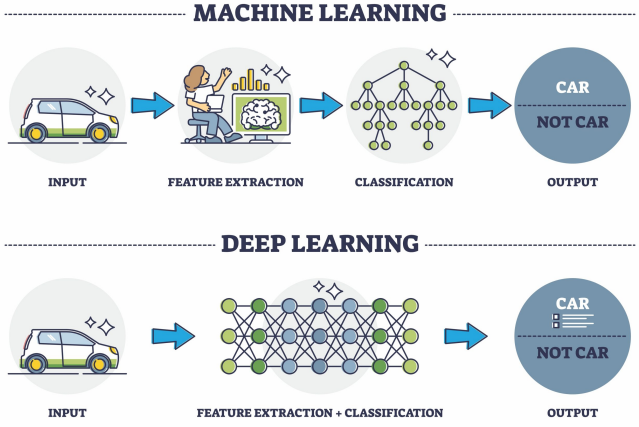
2026. 11. 2.

Artificial Intelligence vs. Machine Learning vs. Deep Learning



KNU
KANGWON NATIONAL UNIVERSITY

Machine Learning vs. Deep Learning

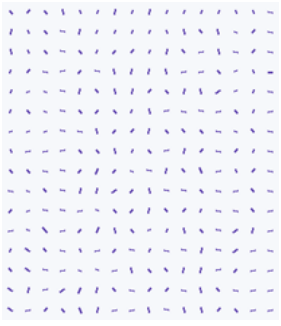


https://www.telliant.com/consider-adding-deep-learning-to-your-next-software-project/

- The machine learning model learns from features selected or designed by humans.

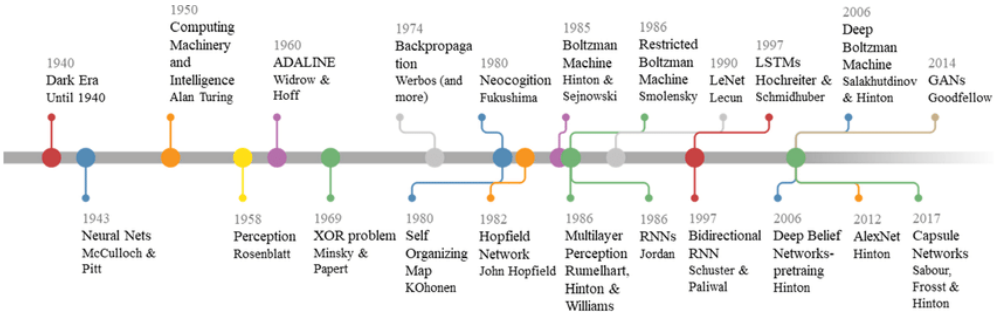


Edges



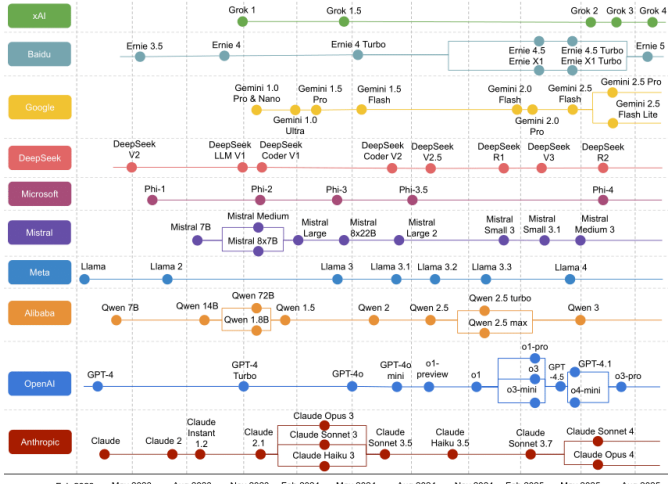
Gradient orientations

Timeline of Deep Learning

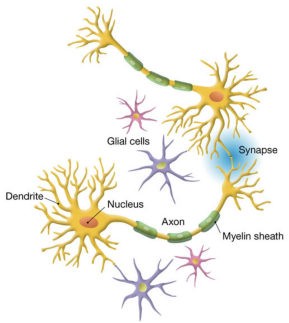


Adapted from Favio Vázquez

Timeline of Large Language Models



Biological Neurons



- Neuron (10^{11}): a special biological cell that processes information
  - Nucleus contains information about hereditary traits
  - Dendrites (receiver) receives signals (impulses) from other neurons
  - Axon (transmitter) transmits signals generated by its cell body
- Synapses (10^{14}): terminals of the strands
  - an elementary structure and functional unit between two neurons
  - neurons have connections to around 10^4 other neurons
  - via certain chemicals (neurotransmitters)

Why (Artificial) Neural Networks?

- Many desirable characteristics of a brain, not present in modern computers
  - Massive parallelism
  - Distributed representation and computation
  - Distributed information across multiple synaptic connections
  - Learning ability
  - Generalization ability
  - Adaptivity
  - Inherent contextual information processing
  - Fault tolerance
  - Low energy consumption

Perceptron

- Originated from attempts to mimic the human brain, which is responsible for intelligent behavior.
- Developed by Frank Rosenblatt in 1957.

Psychological Review
Vol. 65, No. 6, 1958

THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN[1]

F. ROSENBLATT

Cornell Aeronautical Laboratory

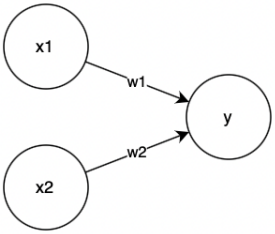
If we are eventually to understand the capability of higher organisms for perceptual recognition, generalization, recall, and thinking, we must first have answers to three fundamental questions:

and the stored pattern. According to this hypothesis, if one understood the code or "wiring diagram" of the nervous system, one should, in principle, be able to discover exactly what an organism remembers by reconstruct-

"If we are eventually to understand the capability of higher organisms for perceptual recognition, generalization, recall, and thinking, we must first have answers to three fundamental question."(우리가 궁극적으로 지각적 인식, 일반화, 기억 그리고 사고를 할 수 있는 고등 유기체의 능력을 이해하기 위해서는 먼저 세 가지 근본적인 질문에 대해 답할 수 있어야 한다.)

1. How is information about the physical world sensed, or detected, by the biological system? (생물 시스템은 실제 물리적 세계에 대한 정보를 어떻게 감지할까?)
2. In what form is information stored, or remembered? (어떤 형태로 저장되거나 기억될까?)
3. How does information contained in storage, or in memory, influence recognition and behavior? (스토리지 또는 메모리에 저장된 정보가 인식과 동작에 어떻게 영향을 미칠까?)

Perceptron



- x_1, x_2: input
- y; output
- w_1, w_2: **weight**

- Serves as the foundation of deep learning.
- Receives multiple input signals and produces a single output signal.
- Unlike biological neurons, it takes binary values (0 = no signal, 1 = signal activated).

$$y = \begin{cases} 0 & (w_1 x_1 + w_2 x_2 \leq \theta) \\ 1 & (w_1 x_1 + w_2 x_2 > \theta) \end{cases}$$

  - A neuron outputs 1 only when the total input signal exceeds the threshold θ.

Basic Logic Gates

- A perceptron can implement logic gates.

AND gate

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

$(w_1, w_2, \theta) = (0.5, 0.5, 0.7)$

NAND gate

x_1	x_2	y
0	0	1
1	0	1
0	1	1
1	1	0

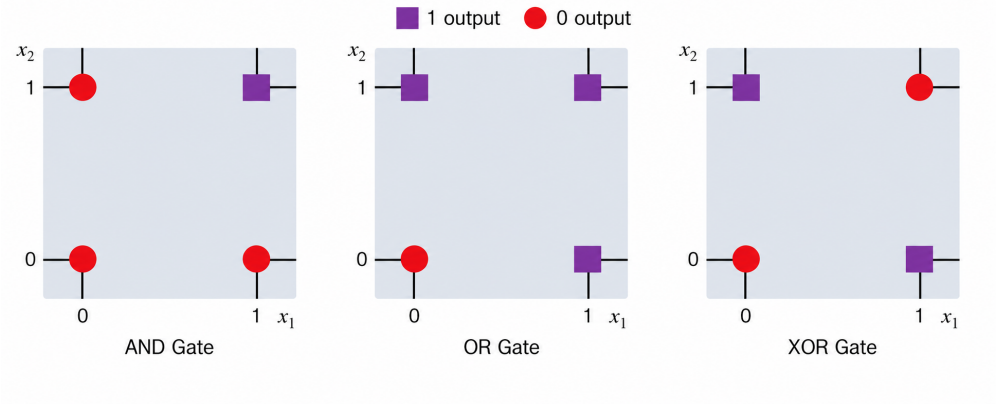
$(w_1, w_2, \theta) = (-0.5, -0.5, -0.7)$

OR gate

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1

$(w_1, w_2, \theta) = (0.5, 0.5, 0.2)$

$\rightarrow$ The structure of perceptrons is identical, but their parameters (weights and thresholds) differ.


1 output
0 output
x_2
1
0
0
1
x_1
AND Gate
OR Gate
XOR Gate

Weights and Bias

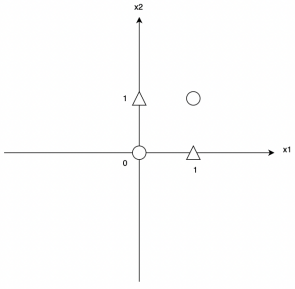
- If we replace θ with $-b$, the equation can be rewritten as follows:

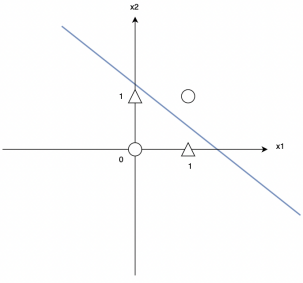
$$y = \begin{cases} 0 & (b + w_1 x_1 + w_2 x_2 \leq 0) \\ 1 & (b + w_1 x_1 + w_2 x_2 > 0) \end{cases}$$

$\rightarrow$ w_1, w_2 are parameters that control the influence (importance) of each input on the output, while the bias (b) is a parameter that determines how easily the neuron is activated.

Limitation of Perceptron

- The perceptron we have seen so far cannot implement the XOR gate.





XOR gate

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

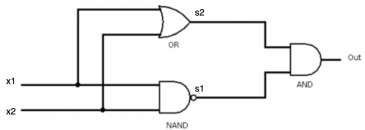
$\rightarrow$ If the constraint of linearity is removed, the data can be separated.

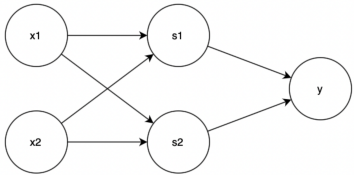
Multi-layer Perceptron

- The XOR gate can be constructed by combining AND, NAND, and OR gates.

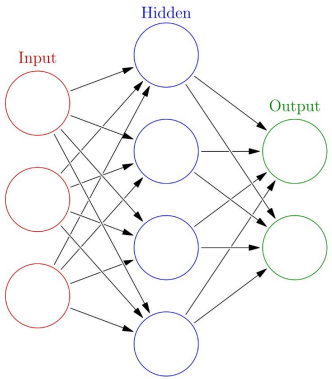
XOR gate

x_1	x_2	s_1	s_2	y
0	0	1	0	0
1	0	1	1	1
0	1	1	1	1
1	1	0	1	0





Perceptron to Artificial Neural Networks



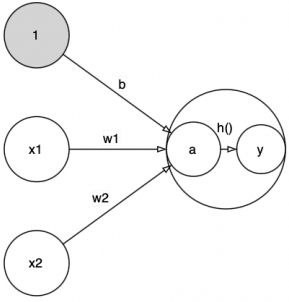
- How are signals transmitted in a neural network?

$$y = h(b + w_1 x_1 + w_2 x_2 + ...)$$

$$h(x) = \begin{cases} 0 & (x \leq 0) \\ 1 & (x > 0) \end{cases}$$

The total sum of input signals is transformed through a function $h(x)$, and the transformed value represents the output y.

Activation Function

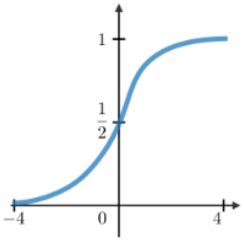
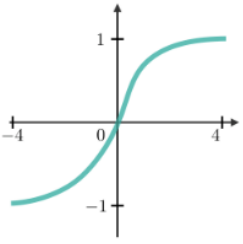
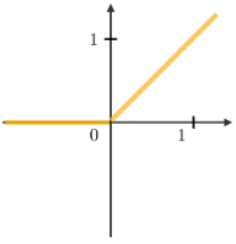


- The activation function determines whether the total sum of input signals triggers activation.

$$a = b + w_1 x_1 + w_2 x_2$$

$$y = h(a)$$

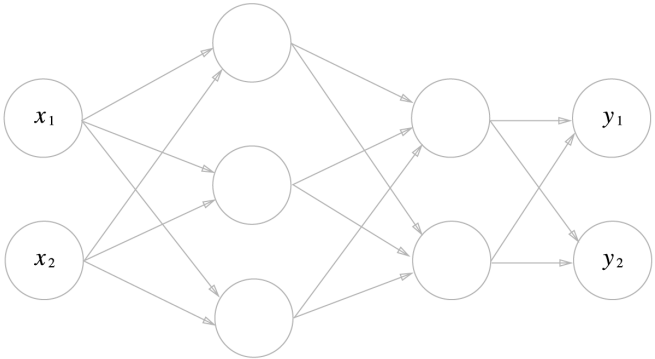
Types of Activation Functions

Sigmoid	Tanh	RELU
$g(z) = \frac{1}{1+e^{-z}}$	$g(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$g(z) = \max(0, z)$
		

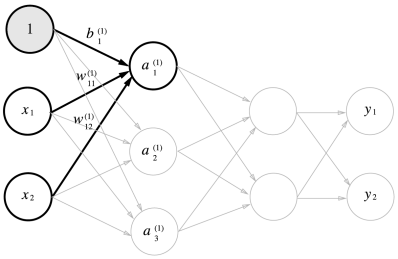
https://studymachinelearning.com/activation-functions-in-neural-network/

3-layer Perceptron

- The input layer consists of 2 nodes, the first hidden layer has 3 nodes, the second hidden layer has 2 nodes, and the output layer consists of 2 nodes.



3-layer Perceptron: Input Layer to First Hidden Layer



$$a_1^{(1)} = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}$$
$$a_2^{(1)} = w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)}$$
$$a_3^{(1)} = w_{31}^{(1)} x_1 + w_{32}^{(1)} x_2 + b_3^{(1)}$$

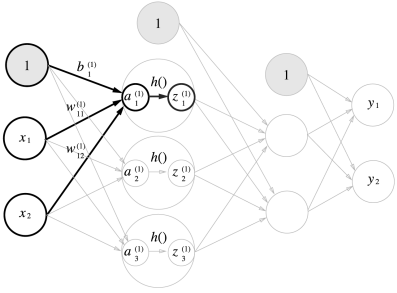
$$\mathbf{a}^{(1)} = \mathbf{x}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}$$

$$\mathbf{X} = (x_1, x_2),$$

$$\mathbf{a}^{(1)} = (a_1^{(1)}, a_2^{(1)}, a_3^{(1)}), \mathbf{b}^{(1)} = (b_1^{(1)}, b_2^{(1)}, b_3^{(1)}),$$

$$\mathbf{W}^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{21}^{(1)} & w_{31}^{(1)} \\ w_{12}^{(1)} & w_{22}^{(1)} & w_{32}^{(1)} \end{pmatrix}$$

3-layer Perceptron: Activation Function



$$\mathbf{z}^{(1)} = h(\mathbf{a}^{(1)})$$

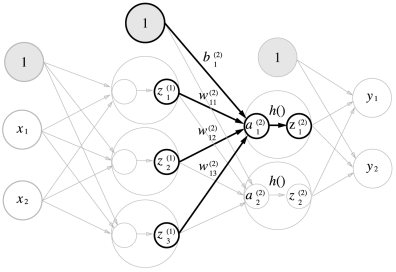
$$h : \mathbb{R}^n \to \mathbb{R}^n$$

$$h(\mathbf{x}) = (h(x_1), h(x_2), \ldots, h(x_j), \ldots, h(x_n))$$

where $\quad \mathbf{x} = (x_1, x_2, \ldots, x_n) \in \mathbb{R}^n$

$$h(x_j) = \frac{1}{1 + e^{-x_j}}$$

3-layer Perceptron: First Hidden Layer to Second Hidden Layer

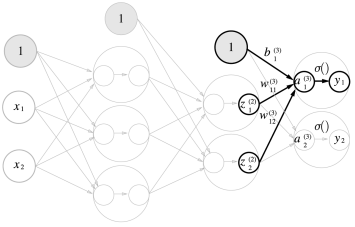


$$\mathbf{a}^{(2)} = \mathbf{z}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$$

$$\mathbf{z}^{(1)} = (z_1^{(1)}, z_2^{(1)}, z_3^{(1)}), \quad \mathbf{W}^{(2)} = \begin{pmatrix} w_{11}^{(2)} & w_{21}^{(2)} \\ w_{12}^{(2)} & w_{22}^{(2)} \\ w_{13}^{(2)} & w_{23}^{(2)} \end{pmatrix},$$

$$\mathbf{b}^{(2)} = (b_1^{(2)}, b_2^{(2)}), \quad \mathbf{a}^{(2)} = (a_1^{(2)}, a_2^{(2)})$$

3-layer Perceptron: Second Hidden Layer to Output Layer



- For regression,
  - Identity function

$$\sigma(x) = x$$

- For classification,
  - Softmax function

$$\sigma : \mathbb{R}^n \to \mathbb{R}^n$$

$$\sigma(\mathbf{x}) = (\sigma(x_1), \sigma(x_2), \ldots, \sigma(x_j), \ldots, \sigma(x_n))$$

$$\sigma(x_j) = \frac{e^{x_j}}{\sum_{i=1}^{n} e^{x_i}}$$

KNU KANGWON NATIONAL UNIVERSITY

Softmax Function

Output layer: $\begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix}$ $\xrightarrow{\text{Softmax activation } \sigma(x_j)=\frac{e^{x_j}}{\sum_{i=1}^{n} e^{x_i}}}$ Probabilities: $\begin{bmatrix} 0.02 \\ 0.90 \\ 0.05 \\ 0.01 \\ 0.02 \end{bmatrix}$

KNU KANGWON NATIONAL UNIVERSITY

Batch Processing

- We estimated the output for a single sample at a time.
  - Each multiplication is dimensionally consistent (matrix dimensions match).

$$\mathbf{x}\ (2) \quad \mathbf{W_1}\ (2 \times 3) \quad \mathbf{W_2}\ (3 \times 2) \quad \mathbf{W_3}\ (2 \times 2) \quad \rightarrow \mathbf{Y}\ (2)$$

- If we process 100 images simultaneously, the output will have a shape of 100×2.

$$\mathbf{X}\ (100 \times 2) \quad \mathbf{W_1}\ (2 \times 3) \quad \mathbf{W_2}\ (3 \times 2) \quad \mathbf{W_3}\ (2 \times 2) \quad \rightarrow \mathbf{Y}\ (100 \times 2)$$

$\rightarrow$ This way of processing multiple data samples at once is called **batch processing**.

Thank You!

(Q & A)

References

- 사이토 고키 (2025), 밑바닥부터 시작하는 딥러닝 1(리마스터판), 한빛미디어